FITTRACK

PROJECT ARCHITECTURE GUIDE

A visual tour of the app, its APIs, data model, and end-to-end flows.

HLD + LLD | ROUTES + APIs | DATA FLOWS

Built from the project README

Next.js 16 | React 19 | TypeScript | Tailwind CSS 4

IMPORTANT: CURRENT APP STATUS

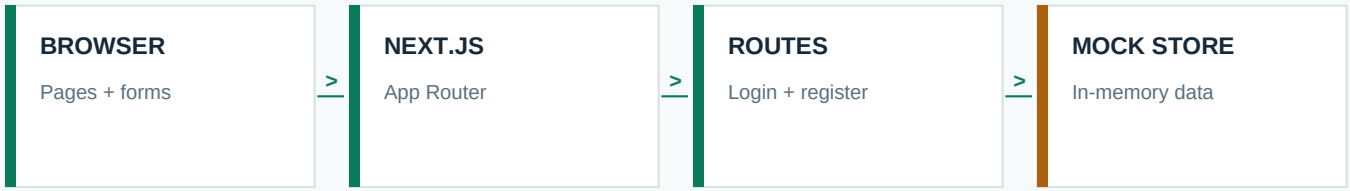
This is a functional UI prototype. Health records and users use in-memory sample data. PostgreSQL/Prisma is described, but not connected to the current page or API data flows.

How FitTrack fits together

Think of the system as a browser UI, route handlers, and demo data.

The big picture

Solid arrows show behavior implemented now. The database connection is only a target design.



The persistence boundary

Prisma describes PostgreSQL tables, but active pages and handlers do not instantiate Prisma Client. Data written to the mock user array is process-local.

Three kinds of runtime work

1. PAGE RENDERING

Feature pages import sample records and render summaries. No health-record API calls.

2. FORM REQUESTS

Login/register send JSON to route handlers. Zod validates incoming payloads.

3. BROWSER THEME

ThemeToggle updates <html>, stores local preference, and uses OS preference by default.

Implemented vs intended

Implemented today	Intended / schema direction
Server-rendered pages with sample data	Persistent per-user records in PostgreSQL
Registration and credential checks in memory	Prisma-backed account and health data
NextAuth handler with JWT option	Forms using real session + protected routes
Browser theme preference	Per-account theme sync (not wired)

Pages and shared UI

A route is a URL. Most health routes share a shell and load demo records.

Public and account pages

URL	What a user sees	Current behavior
/	Landing page	Static feature preview
/login	Sign-in form	POST /api/login; navigate on success
/register	Profile + account form	Register, then call login endpoint
/dashboard	Daily overview	Summary cards, charts, reminders, samples

Health feature pages

URL	Focus	Data / interaction status
/workouts	Sessions and exercises	Sample workouts; no write endpoint
/water	Intake and history	Quick-add is visual only
/sleep	Duration and quality	Sample records and trend
/nutrition	Calories and macros	Sample meals and calculated totals
/activity	Steps and active time	Sample activity logs
/habits	Streaks and completion	Sample habits
/profile	Profile preferences	Sample user; save is not persisted

Navigation is not authorization

Dashboard and feature routes are directly accessible. The app does not currently enforce a signed-in session before rendering them.

Endpoints at a glance

Two custom JSON endpoints and one NextAuth protocol route.

POST /api/register

Zod validates profile fields.
Bcrypt hashes password.
Adds user to in-memory array.

POST /api/login

Validates email/password.
Looks up user; compares bcrypt.
Returns profile summary on success.

Endpoint	Success	Common errors
POST /api/register	200: { ok, user }	400: validation or duplicate
POST /api/login	200: { ok, user }	400 input; 401 credentials; 500 error
NextAuth GET	Session/provider protocol	NextAuth-defined behavior
NextAuth POST	Sign-in/callback/sign-out	NextAuth-defined behavior

Registration request example

```
{
  "name": "Ava Thompson", "email": "ava@example.com",
  "password": "password123", "dateOfBirth": "1996-08-14",
  "height": 170, "initialWeight": 69, "preferredUnits": "metric",
  "goals": ["Build strength"], "activityPreferences": ["Running"]
}
```

Validation: name length >= 2; valid email; password length >= 8; positive height and weight; units metric or imperial.
Goals/preferences default to empty arrays. Date of birth is checked only as a string.

Important authentication distinction

POST /api/login returns JSON only; it does not set a NextAuth JWT session. The forms call this custom endpoint, not the NextAuth sign-in protocol.

What happens after submit?

Follow each step from user action to response.

Registration: two requests, one redirect

1 Submit form
Browser sends profile JSON.

2 Validate
Zod checks request fields.

3 Create demo user
Bcrypt hash; append in memory.

4 Verify login
Form posts credentials again.

5 Navigate
On login 200, go to /dashboard.

! No session cookie
This flow does not create NextAuth session.

Login: credential verification

1 Submit
POST email + password.

2 Validate
Zod checks email and length.

3 Find user
Case-insensitive mock lookup.

4 Compare
bcrypt verifies password.

200 Redirect
Client routes to dashboard.

401 No account / mismatch
Same response avoids account detail leak.

Feature page path

Request page > server component imports sample data > calculates display values > renders shared shell and HTML. No resource API or database is called.

Code layout and responsibilities

Use this map to find where a behavior lives.

Repository map

app/	URL routes, pages, API handlers, root layout
api/login/route.ts	Custom credential verification
api/register/route.ts	Validated demo account creation
api/auth/[...nextauth]/route.ts	NextAuth credentials and JWT config
components/	DashboardShell, ThemeToggle, reusable UI
components/ui/	Button, Card, Input, Label, Badge, etc.
lib/mock-db.ts	Sample records + in-memory user helpers
lib/utils.ts	cn(): merge classes and Tailwind utilities
prisma/schema.prisma	PostgreSQL model definitions; not wired

Component behavior

Unit	Responsibility	Notes
DashboardShell	Shared sidebar/header/mobile nav	Wraps health routes
ThemeToggle	OS preference + localStorage	Sets .dark class on html
UI primitives	Consistent controls and surfaces	Tailwind via cn()
Page components	Read/format sample records	Mostly server components
Login/Register	Client state, fetch, errors, routing	Call custom JSON APIs

Server vs client

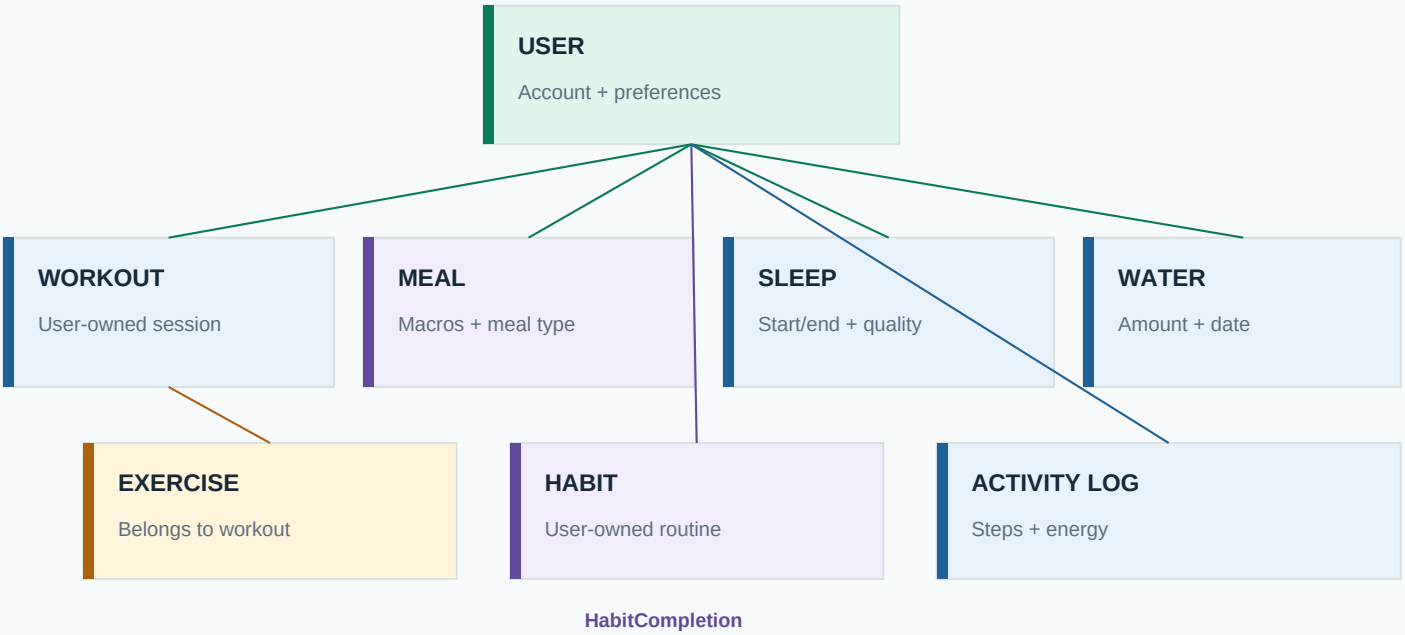
Pages render on the server when they do not need browser state. Forms manage interactive state. ThemeToggle is client-side because it reads localStorage and changes the document.

The planned relational shape

The Prisma schema captures which health records belong to a user.

Entity relationship map

Each user owns their logs. A workout has exercises; a habit has completion records.



Model	Key fields (simplified)	Relationship
User	name, unique email, hash, units, theme	Parent for health records
Workout / Exercise	title, duration / name, sets, reps, weight	Workout has exercises
Meal	calories, protein, carbs, fats, consumedAt	Belongs to user
Sleep / Water	sleep start/end, quality / amount, date	Belongs to user
Habit / Completion	title, frequency / date, completed	Habit has completions
Measurement / Activity	weight, waist / steps, distance, minutes	Belongs to user

Schema != runtime database

PostgreSQL uses DATABASE_URL and models have CUID IDs, but there is no Prisma Client integration or migration flow in the current app.

Run it, then understand its limits

A quick start plus a short path from demo to persistent product.

Local quick-start

```
npm install
npm run dev
```

Open http://localhost:3000

Command	Purpose
npm run dev	Start local development server
npm run build	Build and type-check production app
npm run start	Serve the production build
npm run lint	Run ESLint

Demo account

EMAIL

demo@fittrack.app

PASSWORD

demo123

What is needed before production?

- Replace in-memory users with Prisma-backed PostgreSQL and migrations.
- Connect health pages to authenticated per-user read and write endpoints.
- Use NextAuth sign-in from forms and protect private routes with sessions.
- Set a strong NEXTAUTH_SECRET; remove development fallback and demo credentials.
- Add rate limiting, robust input/error policies, and production monitoring.

Environment variables

Prisma expects DATABASE_URL. NextAuth reads NEXTAUTH_SECRET but currently has a development fallback. Never commit secrets.

Source: project README and current source layout.